

Phase 7: Integration & External Access

AgriConnect

Salesforce-Based Farmer Support & Marketplace System

Step 1: Named Credentials

➤ Introduction

- Named Credentials in Salesforce provide a **secure and reusable way** to store authentication details and endpoint URLs for external systems. Instead of hardcoding usernames, passwords, or access tokens in Apex, developers can simply reference a Named Credential. This ensures **security, easy maintenance, and consistency** across integrations.

➤ Step 1: Identify External Services

- For the AgriConnect project, we may need to connect with multiple external systems such as:
- **Weather API** (<https://api.weather.example.com>) – to fetch weather forecasts for farmers.
- **Government Agriculture API** (<https://api.gov.agriculture.in>) – to validate subsidy schemes and farmer records.
- **Payment Gateway API** (<https://api.payment.example.com>) – to handle secure transactions for crop sales.

➤ Step 2: Create Named Credential

- Navigate to **Setup → Quick Find → Named Credentials → New Named Credential**.
- Provide the following details:
 - **Label:** WeatherAPI_NC

- **Name (API Name):** WeatherAPI_NC
- **URL:** https://api.weather.example.com (only the base URL).
- **Identity Type:** Named Principal (for system-level authentication).
- **Authentication Protocol:** OAuth 2.0 (recommended) or Password Authentication depending on the external API.
- If using OAuth 2.0, enter **Consumer Key, Consumer Secret, Callback URL, and Scopes**.
- Select **Start Authentication Flow on Save** to authenticate immediately.
- Click **Save**.

➤ **Step 3: Test Named Credential**

- Once created, Named Credentials can be tested with Apex code:


```
HttpRequest req = new HttpRequest();
req.setEndpoint('callout:WeatherAPI_NC/v1/forecast?lat=17.3&lon=78.4');
req.setMethod('GET');
Http http = new Http();
HttpResponse res = http.send(req);
System.debug('Response: ' + res.getBody());
```
- If successful, the debug logs will show live data from the external API.

➤ **Step 4: Assign Access with Permission Set**

- Go to **Setup → Permission Sets → New Permission Set**.
 - Label: PS_WeatherAPI_Access
 - API Name: PS_WeatherAPI_Access
- Within the Permission Set, navigate to **Named Credential Access** and enable access to WeatherAPI_NC.

- Assign this permission set to the **Integration User** or relevant administrators.
- **Step 5: Use in Flows**
- Named Credentials can also be used directly in **Flow Builder** without Apex. Developers can add an **Action**, select the Named Credential (WeatherAPI_NC), and map request/response fields to automate processes visually.

Step 2: External Services

- External Services in Salesforce allow us to **connect APIs to Salesforce without writing custom Apex code**. By registering an API's **OpenAPI (Swagger) specification or WSDL**, Salesforce automatically generates **invocable actions** that can be used directly in **Flow Builder** or **Process Automation**. This makes integrations simpler, faster, and more secure.
- **Key Steps**
- Navigate to **Setup → External Services → New External Service**.
 - Provide a **Name** (e.g., AgriSupportService).
 - Upload or enter the URL of the API schema (OpenAPI/Swagger or WSDL file).
 - Select an existing **Named Credential** for authentication (e.g., WeatherAPI_NC).
 - Salesforce will automatically register all API operations and make them available as **actions in Flow**.
 - Use these generated actions in **Flow Builder** to call the external service without writing Apex code.
- **Example for AgriConnect**
- **Weather API:** Farmers can check forecasts via a Flow that calls the Weather API.

- **Government API:** Validate farmer Aadhaar or subsidy eligibility through registered services.
- **Payment Gateway API:** Process payments directly from Salesforce using invocable actions.

Step 3: Web Services (REST/SOAP)

➤ Salesforce can **expose its data and business logic** as **REST or SOAP web services**, allowing external applications (mobile apps, partner portals, or government systems) to interact with Salesforce in real time. REST services are lightweight and easier for modern apps, while SOAP is preferred for legacy systems that require formal contracts (WSDL).

➤ Key Steps

- **Apex REST Services**

- Navigate to **Setup** → **Apex Classes** → **New** (or use Developer Console).
- Create a class with the `@RestResource` annotation.
- Define methods with HTTP annotations like `@HttpGet`, `@HttpPost`, `@HttpPatch`, or `@HttpDelete`.
- External apps will call the endpoint:
`https://<instance>.my.salesforce.com/services/apexrest/<url Mapping>`
- Require authentication via **Connected App OAuth token**.

- **Example:**

- Class name: `AgriOrdersService`
- URL mapping: `/agriconnect/orders/*`
- Purpose: External apps can create or update farmer crop orders.

- **Apex SOAP Services**

- Create a **global Apex class** with webservice static methods.

- Save the class and then click **Generate WSDL**.
- Share the WSDL file with external partners to consume Salesforce SOAP endpoints.
- All methods must use serializable data types (String, Integer, custom objects via JSON if needed).
- **Example:**
 - Class name: AgriSoapService
 - Method: submitOrder(String payloadJson)
 - Purpose: External ERP or legacy systems can submit crop orders via SOAP.

Step 4: Callouts

- Callouts in Salesforce allow the platform to **communicate with external systems in real-time** by sending HTTP requests (REST) or SOAP messages. They are used when Salesforce needs to **fetch data from, or send data to, external services** such as weather APIs, government databases, or payment gateways. Callouts require either **Remote Site Settings** or **Named Credentials** for security.
- **Key Steps**
 - **Prepare Remote Site / Named Credential**
 - Go to **Setup** → **Remote Site Settings** → **New** (or use Named Credentials).
 - Add the external endpoint URL to authorize Salesforce to make callouts.
 - For secure authentication, create a **Named Credential** instead of hardcoding credentials in Apex.
 - **Create Apex Callout Class**
 - Navigate to **Setup** → **Apex Classes** → **New**.
 - Create a class using the HttpRequest and Http classes.

- Set the endpoint using callout:<Named_Credential_Name> syntax.
- Send GET, POST, PUT, or DELETE requests as required.
- **Example:**

```
HttpRequest req = new HttpRequest();
req.setEndpoint('callout:WeatherAPI_NC/v1/forecast?lat=17.3&lon=78.4');
req.setMethod('GET');
Http http = new Http();
HttpResponse res = http.send(req);
System.debug('Weather Response: ' + res.getBody());
```
- **Test Callouts Using HttpCalloutMock**
 - Salesforce requires that **all callouts in tests are mocked**.
 - Implement the HttpCalloutMock interface to simulate API responses.
 - Use Test.setMock() in your test classes to test the callout logic without hitting real external APIs.
- **Use Callouts in Flows or Processes**
 - Callouts can also be used via **Invocable Apex** methods, allowing Flows and Process Builder to execute real-time requests to external systems.

Step 5: Platform Events

- Platform Events in Salesforce are **custom events** used for **real-time communication** within Salesforce or with external systems. They allow **publish-subscribe architecture**, meaning one process can **publish an event**, and multiple subscribers (Flows, Apex triggers, external systems) can **respond to it immediately**. In AgriConnect, Platform Events are ideal for notifying farmers, admins, or marketplaces when orders are created, updated, or completed.

➤ Key Steps

- **Create a Platform Event**

- Go to **Setup** → **Platform Events** → **New Platform Event**.
- Enter **Label:** OrderUpdate__e
- API Name: OrderUpdate__e
- Set **Publish Behavior:** Publish Immediately (for real-time delivery).
- Add custom fields relevant to the event:
 - OrderId__c (Text)
 - Status__c (Text)
 - Message__c (Long Text Area)
- Save the Platform Event.

- **Publish Events in Apex or Flow**

- **Apex Example:**

```
OrderUpdate__e event = new OrderUpdate__e(  
    OrderId__c = 'a0X1x00000XYZ123',  
    Status__c = 'Shipped',  
    Message__c = 'Your order is on the way'  
);  
EventBus.publish(event);
```

- **Flow Example:** Use “**Create Records**” action → select **Platform Event** as the object → map fields.

- **Subscribe to Platform Events**

- Internal Salesforce: Create a **Platform Event-Triggered Flow** to update records or send notifications.
- External Systems: Use **CometD / Pub/Sub API** to receive real-time events outside Salesforce.

- **Use Cases in AgriConnect**

- Notify farmers when **crop orders are confirmed or shipped**.

- Alert marketplace admins when **new orders are created**.
- Trigger **external system workflows** (like delivery tracking or ERP updates) via subscribed events.

Step 6: Change Data Capture

- Change Data Capture (CDC) in Salesforce enables **real-time monitoring of record changes** (create, update, delete, undelete, merge) on standard and custom objects. It allows both **internal processes and external systems** to stay synchronized with Salesforce data without polling the database repeatedly.
- **Key Steps**
 - Navigate to **Setup → Change Data Capture**.
 - Select the objects to monitor, e.g., Order__c, Inventory__c, Farmer__c.
 - Click **Save**.
 - External systems can subscribe to these events via **CometD / Pub/Sub API**.
 - Internal Salesforce subscribers can use **Platform Event-triggered Flows** or **Apex triggers**.
- **Use Cases in AgriConnect:**
 - Sync updated crop orders with an ERP system.
 - Notify marketplaces of inventory changes.
 - Track farmer data updates in real-time.

Step 7: Salesforce Connect

- Salesforce Connect allows access to **external data sources** in real-time without storing the data inside Salesforce. External objects behave like standard/custom objects but fetch data **on-demand**.
- **Key Steps**
 - Navigate to **Setup → External Data Sources → New External Data Source**.

- Fill details:
 - **Label / Name:** ERP_Inventory_ExternalDS
 - **Type:** OData 4.0
 - **URL:** https://erp.example.com/odata/
 - **Identity Type:** Named Principal using a Named Credential (NC_ERP).
- Click **Validate & Sync** → select tables (e.g., Inventory, MarketRates).
- External objects are now accessible as ERP_Inventory__x in Salesforce UI, Flows, and Reports.
- **Use Cases in AgriConnect:**
 - View ERP inventory data directly in Salesforce.
 - Show market rates to farmers in real-time without replication.

Step 8: API Limits

- Salesforce imposes **API limits** to ensure fair resource usage across all orgs. Every integration (REST, SOAP, Bulk API, Platform Events, External Services) consumes API calls. Monitoring limits prevents **service disruption**.
- **Key Steps**
 - Go to **Setup** → **System Overview** or **Company Information** to check usage.
 - Use **REST resource:** /services/data/vXX.X/limits to programmatically monitor.
 - Design integrations to **batch requests** and **use caching** to avoid exceeding limits.
 - Monitor daily and weekly to ensure external integrations like AgriConnect API calls stay within limits.

Step 9: OAuth & Authentication

- OAuth is the **secure method for external applications** to access Salesforce data. Salesforce uses **Connected Apps** to manage authentication, issuing **access tokens** for API requests.

- **Key Steps**

- Navigate to **Setup → App Manager → New Connected App**.
- Fill details:
 - **Name:** AgriConnect_MobileApp
 - **Enable OAuth Settings** → **Callback URL:**
`https://agriconnect.example.com/oauth/callback`
 - Select **OAuth Scopes:** api, refresh_token, offline_access
- Save and copy **Consumer Key & Secret**.
- External apps use these to get access tokens for calling Salesforce APIs.
- Can be used with **Named Credentials** in Apex to simplify authentication.

Step 10: Remote Site Settings

- Remote Site Settings authorize Salesforce to **call external endpoints** via HTTP. Without this, Apex callouts or integrations will fail for security reasons.

- **Key Steps**

- Navigate to **Setup → Remote Site Settings → New Remote Site**.
- Fill details:
 - **Remote Site Name:** Remote_Agri_API
 - **Remote Site URL:** Base URL of the external service (e.g.,
`https://api.weather.example.com`)
 - **Description:** Allows Salesforce callouts to external Agri APIs
 - Check **Active** → **Save**
- Repeat for each external host if multiple APIs are used.

- **Note:** If a **Named Credential** is used, Remote Site Setting is not strictly required.

Conclusion

- Phase 7 of AgriConnect focuses on enabling **secure, real-time, and scalable integrations** between Salesforce and external systems such as weather APIs, government databases, payment gateways, and ERP systems. By implementing **Named Credentials and Remote Site Settings**, all callouts are authorized and secured. **External Services and Apex Web Services (REST/SOAP)** provide flexible ways to connect and expose Salesforce data to other applications.
- The use of **Platform Events** and **Change Data Capture (CDC)** ensures that both internal users and external systems receive **instant updates** on critical operations such as crop orders, inventory changes, and farmer records. **Salesforce Connect** allows seamless access to external data sources without storing the data in Salesforce, while proper monitoring of **API Limits** and the use of **OAuth authentication via Connected Apps** ensures **secure, controlled, and efficient access** to Salesforce resources.
- Overall, these integrations provide AgriConnect with a **robust, real-time, and maintainable framework**, improving operational efficiency, farmer engagement, and marketplace responsiveness while maintaining security and scalability for future enhancements.